LEARN BEING FORWARD

Programming — Python Programming

Python syntax & data types

COMPLETE LEARNING GUIDE

What You'll Learn

Python's defining trait is that whitespace and a small, consistent grammar carry the structure other languages express with braces and semicolons — which makes code read almost like pseudocode, but also means small indentation mistakes become real bugs instead of style complaints. This sub-module covers the mechanics every later topic assumes: how indentation defines blocks, how variables bind dynamically to objects rather than declaring a fixed type, the core built-in types (int, float, bool, str, None) and how Python decides what's truthy, and the string and formatting tools (slicing, f-strings) used constantly in real code. Getting comfortable with dynamic typing and Python's particular truthiness rules early prevents a long tail of confusing bugs later, especially for anyone arriving with a background in a statically-typed language where these rules work differently.

1. Indentation, Blocks, and Statements

Python uses indentation, not braces, to delimit code blocks — every line inside an if, for, while, def, or class body must be indented consistently (4 spaces is the near-universal convention per PEP 8), and mixing tabs and spaces, or indenting inconsistently within the same block, raises an IndentationError. A colon at the end of a header line (if x:, def f():) always introduces an indented block beneath it. Statements normally end at a newline with no terminator needed, though a backslash or enclosing brackets allow a logical line to span multiple physical lines. Comments start with # and run to the end of the line; there is no single-line block-comment syntax, though a triple-quoted string left unassigned is commonly used as a multi-line docstring or comment.

- A mix of tabs and spaces in the same file is a common source of indentation errors that look fine in one editor but fail in another.
- PEP 8 (the official style guide) recommends 4 spaces per indentation level and no tabs — most tooling defaults to this.

```python
# Blocks are defined purely by indentation level
age = 20
if age >= 18:
    print("Adult")
    if age >= 65:
        print("Senior")
else:
    print("Minor")

# A docstring as the first statement in a function documents it
def greet(name: str) -> str:
    """Return a friendly greeting for `name`."""
    return f"Hello, {name}!"
```

2. Variables and Dynamic Typing

A Python variable is just a name bound to an object in memory — there is no type declaration, and the same name can be rebound to a value of a completely different type later (x = 5 then x = "five" is legal, if usually a sign of poor naming). The type lives on the object, not the name: type(x) tells you what x currently refers to. Multiple names can be assigned in one line (a, b = 1, 2) and swapped without a temporary variable (a, b = b, a) because the right-hand side is fully evaluated into a tuple before any assignment happens. Since Python 3.6, optional type hints (name: str = "Ana") can annotate intent for tooling and readers, but they are not enforced at runtime — they are purely advisory.

- Type hints (x: int) are checked by external tools like mypy, not by the Python interpreter itself at runtime.
- a, b = b, a is the idiomatic Python swap — no temporary variable needed.

```python
x = 5
print(type(x))      # <class 'int'>
x = "five"
print(type(x))      # <class 'str'> — same name, new type, nothing enforced

a, b = 1, 2
a, b = b, a          # swap: a is now 2, b is now 1
print(a, b)          # 2 1
```

3. Numeric Types and Arithmetic Operators

Python has three built-in numeric types: int (arbitrary precision — Python ints never overflow), float (double-precision), and complex (rarely used outside scientific code). Standard arithmetic operators (+, -, *, /) work as expected, but / always returns a float even for two ints (7 / 2 == 3.5), while // performs floor (integer) division (7 // 2 == 3) and % returns the remainder. ** is the exponent operator (2 ** 10 == 1024), and because Python ints have no fixed size, 2 ** 200 computes exactly with no overflow, unlike fixed-width integer types in many other languages.

- / always returns a float, even 4 / 2 == 2.0 — use // when you specifically need integer division.
- Python ints are arbitrary-precision, so there is no int overflow to worry about, unlike C, Java, or (unboxed) languages with fixed-width integers.

```python
print(7 / 2)     # 3.5  (true division, always float)
print(7 // 2)    # 3    (floor division)
print(7 % 2)     # 1    (remainder)
print(2 ** 10)   # 1024 (exponentiation)
print(2 ** 200)  # exact, arbitrarily large — no overflow
```

4. Strings: Immutability, Slicing, and f-strings

Strings are immutable sequences of characters — every 'modification' (upper(), replace(), concatenation) returns a brand-new string rather than changing the original in place. Slicing (s[start:stop:step]) extracts a substring without mutating s, and negative indices count from the end (s[-1] is the last character). f-strings (introduced in 3.6, now the standard way to format text) embed expressions directly inside {} braces, including format specifiers for decimals, padding, and alignment, and are both faster and more readable than the older % or str.format() styles.

- s[::-1] reverses a string via slicing — a common idiom, not a dedicated reverse() method (strings have no such method).
- f"{value:.2f}" formats a float to exactly two decimal places directly inside an f-string.

```python
name = "Alice"
price = 19.999

print(name[0])        # 'A'
print(name[-1])       # 'e'
print(name[1:4])      # 'lic'
print(name[::-1])     # 'ecilA'

print(f"{name} owes ${price:.2f}")  # Alice owes $20.00
```

5. Booleans, None, and Truthiness

bool is technically a subclass of int (True == 1 and False == 0 both hold), which is why booleans can be summed directly to count True values. None is Python's singleton 'absence of a value,' distinct from False, 0, or an empty string — always compare against it with is None, never == None, since is checks identity rather than relying on (possibly overridden) equality. In a boolean context (if, while, bool(x)), every object is truthy except a specific list of falsy values: False, None, 0, 0.0, and any empty collection ("", [], {}, set()) — this lets idioms like if not my_list: double as both an existence check and an emptiness check.

- Use `x is None` / `x is not None`, never `x == None` — is checks identity, which is the correct, idiomatic test for the None singleton.
- sum([True, True, False, True]) == 3, since booleans behave as 1/0 in arithmetic contexts.

```python
value = None
if value is None:
    print("No value provided")

empty_list = []
if not empty_list:
    print("List is empty")   # falsy: empty collections are falsy

scores = [True, False, True, True]
print(sum(scores))  # 3
```

6. Type Conversion and Checking

Built-in functions int(), float(), str(), bool(), and list() convert a value to that type where a sensible conversion exists — int("42") works, but int("abc") raises a ValueError, so converting unvalidated input should be wrapped in a try/except. type(x) returns an object's exact class, while isinstance(x, SomeType) is almost always preferable for checks because it also respects inheritance (a bool passed to isinstance(x, int) returns True, since bool subclasses int) and accepts a tuple of types to check against several at once.

- isinstance(x, (int, float)) checks against multiple types in one call — cleaner than chaining `or`.
- int("3.5") raises ValueError; convert through float first — int(float("3.5")) — if truncation is intended.

```python
raw = "42"
try:
    number = int(raw)
except ValueError:
    number = None

print(isinstance(True, int))        # True — bool is a subclass of int
print(isinstance(3.5, (int, float)))  # True
```

7. The Walrus Operator and Expression Assignment

Introduced in Python 3.8, the walrus operator (:=) assigns a value to a name as part of a larger expression, which is especially useful for avoiding a redundant second computation inside a while loop condition or a comprehension filter. It's syntax sugar for readability and efficiency, not a new capability — anything written with := could be rewritten with an extra line of plain assignment, but the inline form keeps related logic visually together.

- Most useful in while loops reading input/data, and in list comprehensions that filter on a computed value without recomputing it.

```python
data = [1, 4, 9, 16, 25, 36]

# Keep values whose square root is a whole number greater than 3
results = [root for n in data if (root := n ** 0.5) > 3 and root.is_integer()]
print(results)  # [16.0, 25.0, 36.0]
```

Common Pitfalls

- Mixing tabs and spaces for indentation, producing errors that may not even show up consistently across editors.
- Using == None instead of is None to check for Python's None singleton.
- Assuming / performs integer division like in some other languages — in Python 3, / always returns a float; use // for floor division.
- Forgetting that strings are immutable, writing `s.upper()` on its own line and expecting `s` itself to change.
- Treating an empty string, 0, or an empty list as equivalent to None, rather than using explicit `is None` checks when None specifically matters.
- Calling int() or float() directly on unvalidated user input without a try/except, crashing the program on a ValueError.

Key Takeaways

- Indentation is syntactically significant in Python — it replaces braces as the way blocks are delimited.
- Variables are names bound to objects; the type lives on the object, and the same name can be rebound to a different type.
- / always returns a float; // performs floor division; Python ints are arbitrary-precision and never overflow.
- Strings are immutable — every string method returns a new string rather than mutating in place.
- Falsy values are False, None, 0, 0.0, and empty collections — everything else is truthy in a boolean context.
- Prefer isinstance() over type() for type checks, since it correctly handles inheritance (e.g., bool is a subclass of int).

Learn More

Python Docs — The Python Tutorial: An Informal Introduction
https://docs.python.org/3/tutorial/introduction.html

Python Docs — Built-in Types
https://docs.python.org/3/library/stdtypes.html

PEP 8 — Style Guide for Python Code
https://peps.python.org/pep-0008/